DRIFTLESS — End-to-End System Architecture &

Workflows

1. System Architecture Map

```
graph TB
    subgraph BrowserClient["User Browser Runtime (Google Chrome)"]
        subgraph ExtensionRuntime["Driftless Extension (Manifest V3)"]
            POPUP["Popup Controller<br/>(popup.html / popup.js)"]
            BG["Background Service Worker<br/>(background.js)"]
            BLOCKED_PAGE["Blocked Landing Page<br/>(blocked.html)"]
        end

        subgraph ChromeAPIs["Native Chrome APIs"]
            API_TABS["chrome.tabs<br/>(URL & switch events)"]
            API_WINDOWS["chrome.windows<br/>(Focus state)"]
            API_IDLE["chrome.idle<br/>(60s inactivity)"]
            API_NOTIF["chrome.notifications<br/>(Nudges & alerts)"]
            API_ALARMS["chrome.alarms<br/>(Periodic sync & midnight)"]
            API_STORAGE["chrome.storage.local<br/>(siteTimes & unsyncedDeltas)"]
            API_DNR["declarativeNetRequest<br/>(Dynamic redirect rules)"]
        end

        subgraph WebClient["Web Dashboard Application"]
            AUTH_VIEW["Auth & Persona Portal<br/>(auth.html)"]
            DASH_VIEW["Analytics Dashboard<br/>(dashboard.html)"]
            CHART_ENGINE["Chart.js 4.4 Engine<br/>(Canvas visualization)"]
        end
    end

    subgraph BackendServer["Django 4.2 LTS Application Server (port 8000)"]
        MIDDLEWARE["Middleware Stack<br/>(CORS, WhiteNoise, Session)"]
        ROUTER["URL Dispatcher<br/>(driftless/urls.py & core/urls.py)"]

        subgraph DRFControllers["Django REST Framework Views"]
            AUTH_CTRL["Auth Controller<br/>(/api/auth/*)"]
            USAGE_CTRL["Usage Ingestion<br/>(/api/usage/)"]
            SUMMARY_CTRL["Analytics Engine<br/>(/api/summary/)"]
            LIMITS_CTRL["Limits Controller<br/>(/api/limits/)"]
            SETTINGS_CTRL["Settings Controller<br/>(/api/settings/)"]
            CATEGORY_CTRL["Category Controller<br/>(/api/categories/)"]
            PRIVACY_CTRL["Data Sovereignty<br/>(/api/data/*)"]
        end

        ORM["Django ORM Layer"]
    end

    subgraph Database["Persistence Layer (SQLite)"]
        SQLITE_DB[("db.sqlite3<br/>Tables: User, UserProfile,<br/>UserSettings, UsageLog,
<br/>SiteLimit, CategoryOverride,<br/>FocusSession, authtoken_token")]
    end

    %% Extension Internal Wiring
```

```
    BG ⟶ API_TABS
    BG ⟶ API_WINDOWS
    BG ⟶ API_IDLE
    BG ⟶ API_NOTIF
    BG ⟶ API_ALARMS
    BG ⟶ API_STORAGE
    BG ⟶ API_DNR
    POPUP ⟶ API_STORAGE
    POPUP -- "chrome.runtime.sendMessage" ⟶ BG
    API_DNR -- "Redirects violation" ⟶ BLOCKED_PAGE

    %% Extension to Backend
    BG -- "POST /api/usage/ (Bearer Token)" ⟶ MIDDLEWARE
    BG -- "GET /api/limits/ & /api/settings/" ⟶ MIDDLEWARE
    POPUP -- "PATCH /api/settings/ & /api/limits/" ⟶ MIDDLEWARE

    %% Web Frontend to Backend
    AUTH_VIEW -- "POST /api/auth/login/ & register/" ⟶ MIDDLEWARE
    DASH_VIEW -- "GET /api/summary/" ⟶ MIDDLEWARE
    DASH_VIEW -- "PATCH /api/settings/ & /api/limits/" ⟶ MIDDLEWARE
    DASH_VIEW ⟶ CHART_ENGINE

    %% Backend Routing & Execution
    MIDDLEWARE ⟶ ROUTER
    ROUTER ⟶ AUTH_CTRL
    ROUTER ⟶ USAGE_CTRL
    ROUTER ⟶ SUMMARY_CTRL
    ROUTER ⟶ LIMITS_CTRL
    ROUTER ⟶ SETTINGS_CTRL
    ROUTER ⟶ CATEGORY_CTRL
    ROUTER ⟶ PRIVACY_CTRL

    AUTH_CTRL ⟶ ORM
    USAGE_CTRL ⟶ ORM
    SUMMARY_CTRL ⟶ ORM
    LIMITS_CTRL ⟶ ORM
    SETTINGS_CTRL ⟶ ORM
    CATEGORY_CTRL ⟶ ORM
    PRIVACY_CTRL ⟶ ORM

    ORM ⟶ SQLITE_DB
```

2. Core Workflow Diagrams

2.1. Website Time Tracking Workflow

```
sequenceDiagram
    autonumber
    actor User
    participant Tabs as chrome.tabs
    participant Idle as chrome.idle
    participant SW as background.js
    participant Storage as chrome.storage.local

    User->>Tabs: Navigates to or switches tab
    Tabs->>SW: onActivated or onUpdated event
    SW->>SW: commitActiveTime()
    Note over SW: If currentTabStartTime exists and window is focused:<br/>elapsed =
floor((now - currentTabStartTime)/1000)
    SW->>Storage: siteTimes[domain] += elapsed
    SW->>Storage: unsyncedDeltas[domain] += elapsed
    SW->>Tabs: Query active tab URL
    Tabs-->>SW: Return new tab URL
    SW->>SW: cleanDomain(url) -> targetDomain
    SW->>SW: currentTabDomain = targetDomain
    SW->>SW: currentTabStartTime = Date.now()

    User->>Idle: Leaves desk for 75 seconds
    Idle->>SW: onStateChanged("idle")
    SW->>SW: commitActiveTime() (Commits up to 60s)
    SW->>SW: isUserActive = false (Pauses tracking)

    User->>Idle: Moves mouse
    Idle->>SW: onStateChanged("active")
    SW->>SW: isUserActive = true
    SW->>SW: currentTabStartTime = Date.now() (Resumes tracking)
```

2.2. Threshold Notifications & 3-Tier Escalation Workflow

```
sequenceDiagram
    autonumber
    actor User
    participant SW as background.js
    participant Storage as chrome.storage.local
    participant Notif as chrome.notifications
    participant DNR as declarativeNetRequest
    participant Tab as Active Browser Tab

    SW->>SW: evaluateSiteLimits(domain, totalSeconds)
    SW->>Storage: Read siteLimits[domain] & escalationState[domain]

    alt totalSeconds ≥ 80% limit and !state.nudged
        SW->>Storage: escalationState[domain].nudged = true
        SW->>Notif: create("nudge-<domain>", title="Nudge ⏳", message="Used 80% ...")
        Notif-->>User: Desktop Toast Nudge Appears
    else totalSeconds ≥ 100% limit and !state.alerted
        SW->>Storage: escalationState[domain].alerted = true, streakAtRisk = true
        SW->>Notif: create("alert-<domain>", title="100% Limit Reached ⚠️", message="5m
grace active ...")
        Notif-->>User: Desktop Toast Warning Appears
    else totalSeconds ≥ 100% limit + 300s (5m Grace) and !state.blocked
        SW->>Storage: escalationState[domain].blocked = true
        SW->>DNR: updateDynamicRules({ addRules: [redirectRule] })
        SW->>Tab: tabs.update(activeTab.id, { url: blocked.html?site=domain })
        Tab-->>User: Displays blocked.html Splash Screen
    end
```

2.3. Focus Mode Activation & Network Blocking Workflow

```
sequenceDiagram
    autonumber
    actor User
    participant Popup as popup.js
    participant SW as background.js
    participant DNR as declarativeNetRequest
    participant Alarms as chrome.alarms
    participant API as Django REST API

    User->>Popup: Clicks "45m" Focus Sprint Button
    Popup->>SW: sendMessage({ type: 'START_FOCUS_TIMER', durationMinutes: 45 })
    SW->>DNR: updateDynamicRules(Install redirect rules for distracting sites)
    SW->>Alarms: create("driftless_focus_timer_end", { delayInMinutes: 45 })
    SW-->>Popup: { success: true }
    Popup->>Popup: renderFocusTimerCountdown(Active: 45:00)

    Note over User,SW: 45 Minutes Pass...

    Alarms->>SW: onAlarm("driftless_focus_timer_end")
    SW->>DNR: updateDynamicRules(removeRuleIds: [focusRules])
    SW->>API: POST /api/sessions/ { duration_minutes: 45 }
    SW->>User: Desktop Notification ("Focus Session Completed! 🎯")
```

2.4. Usage Data Synchronization Workflow

```
sequenceDiagram
    autonumber
    participant Alarms as chrome.alarms
    participant SW as background.js
    participant Storage as chrome.storage.local
    participant API as Django (/api/usage/)
    participant DB as SQLite (UsageLog)

    Alarms->>SW: onAlarm("driftless_periodic_sync") (Every 60s)
    SW->>SW: commitActiveTime()
    SW->>Storage: Read unsyncedDeltas & apiToken

    alt unsyncedDeltas has values > 0 and apiToken present
        SW->>API: POST /api/usage/ [{ website_url, time_spent_seconds, date }]
        API->>DB: Atomically find or create UsageLog rows and increment seconds
        API-->>SW: HTTP 200 OK { created: 0, updated: 2 }
        SW->>Storage: Clear unsyncedDeltas, update lastSyncedTime
        SW->>API: GET /api/settings/ & GET /api/limits/
        API-->>SW: Return fresh server limits & flags
        SW->>Storage: Update siteLimits & focusModeActive in local cache
    else Offline / Server Unreachable
        API--xSW: Network Error
        SW->>SW: Keep unsyncedDeltas intact in storage for next retry
    end
```